

LAPORAN PRAKTIKUM DEEP LEARNING

**LONG SHORT-TERM MEMORY (LSTM) UNTUK
PREDIKSI DERET WAKTU**



Disusun oleh:

HABIBI PUTRA RIZQULLAH

NIM 2411531001

Kelas B

Dosen Pengampu:

Dr. Derisma, S.T., M.T.

**PROGRAM STUDI INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
2026**

DAFTAR ISI

DAFTAR ISI.....	i
DAFTAR GAMBAR	iii
DAFTAR TABEL	iv
BAB I PENDAHULUAN	1
1.1 Latar Belakang	1
1.2 Rumusan Masalah	1
1.3 Tujuan Praktikum.....	2
1.4 Manfaat Praktikum.....	2
BAB II LANDASAN TEORI	3
2.1 Data Sekuensial dan Prediksi Deret Waktu.....	3
2.2 Recurrent Neural Network	3
2.3 Long Short-Term Memory	3
2.4 Gerbang pada LSTM.....	4
2.5 Pembentukan Sequence dan Pembagian Data	4
2.6 Fungsi Loss dan Optimizer Adam.....	4
2.7 Hidden Size dan Jumlah Layer.....	5
2.8 Deret Sinus dan Cosinus	5
BAB III METODOLOGI DAN LANGKAH PRAKTIKUM	6
3.1 Alat dan Bahan.....	6
3.2 Alur Pengerjaan.....	6
3.3 Praktikum Prediksi Deret Sinus	6
3.3.1 Persiapan Library dan Perangkat.....	6
3.3.2 Pembuatan Data Sinus.....	7
3.3.3 Pembentukan Sequence dan Pembagian Data	8
3.3.4 Definisi Model RNN dan LSTM.....	9
3.3.5 Fungsi Pelatihan dan Evaluasi.....	10
3.3.6 Pelatihan RNN dan LSTM Dasar	11
3.3.7 Visualisasi Training Loss	12
3.3.8 Prediksi Data Testing Sinus	13
3.3.9 Eksperimen Hidden Size	14
3.3.10 Visualisasi Eksperimen Hidden Size.....	16
3.3.11 Ringkasan Hasil Praktikum.....	16
3.4 Tugas Praktikum Prediksi Deret Cosinus.....	17
3.4.1 Pembuatan Data Cosinus.....	17
3.4.2 Pembentukan Sequence Data Cosinus	18
3.4.3 Konfigurasi dan Pelatihan Tiga Model LSTM.....	18
3.4.4 Penyusunan Tabel Hasil.....	19
3.4.5 Perbandingan Training Loss dan MSE Testing.....	20
3.4.6 Prediksi Model Terbaik.....	21
3.4.7 Analisis Pengaruh Penambahan Layer.....	22
BAB IV HASIL DAN PEMBAHASAN.....	24
4.1 Hasil Praktikum Deret Sinus	24
4.1.1 Hasil RNN dan LSTM Dasar	24
4.1.2 Perbandingan Hidden Size	25
4.1.3 Analisis Loss dan Prediksi	25
4.2 Hasil Tugas Praktikum Deret Cosinus	26
4.2.1 Hasil Eksperimen	26
4.2.2 Pengaruh Penambahan Layer	26
4.3 Perbandingan LSTM dan RNN	27

4.4 Jawaban Pertanyaan Reflektif	27
4.4.1 Peran Cell State dalam LSTM.....	27
4.4.2 Alasan LSTM Lebih Mampu Mengingat Konteks Jangka Panjang	27
4.4.3 Dampak Penambahan Jumlah Layer	28
BAB V PENUTUP	29
5.1 Kesimpulan	29
DAFTAR PUSTAKA	30

DAFTAR GAMBAR

Gambar 3.1 Import library dan penentuan perangkat.....	7
Gambar 3.2 Visualisasi data deret sinus.....	8
Gambar 3.3 Pembentukan sequence dan pembagian data sinus.....	9
Gambar 3.4 Definisi model RNN dan LSTM	10
Gambar 3.5 Fungsi pelatihan model.....	11
Gambar 3.6 Fungsi evaluasi model	11
Gambar 3.7 Hasil pelatihan RNN dan LSTM dasar	12
Gambar 3.8 Grafik perbandingan training loss RNN dan LSTM.....	13
Gambar 3.9 Perbandingan data asli dan prediksi pada data testing sinus.....	14
Gambar 3.10 Kode eksperimen hidden size	15
Gambar 3.11 Tabel hasil eksperimen hidden size	15
Gambar 3.12 Grafik loss dan MSE testing untuk setiap hidden size.....	16
Gambar 3.13 Ringkasan hasil praktikum	17
Gambar 3.14 Visualisasi data deret cosinus	18
Gambar 3.15 Pembentukan sequence data cosinus	18
Gambar 3.16 Konfigurasi dan pelatihan tiga model LSTM	19
Gambar 3.17 Penyimpanan hasil eksperimen ke DataFrame	20
Gambar 3.18 Tabel hasil eksperimen data cosinus.....	20
Gambar 3.19 Grafik loss dan MSE testing pada data cosinus.....	21
Gambar 3.20 Prediksi model terbaik pada data cosinus	22
Gambar 3.21 Analisis pengaruh penambahan layer	23

DAFTAR TABEL

Tabel 4.1 Hasil perbandingan RNN dan LSTM pada data sinus	24
Tabel 4.2 Hasil eksperimen hidden size pada LSTM satu layer (data sinus)	25
Tabel 4.3 Hasil eksperimen LSTM pada data cosinus	26

BAB I

PENDAHULUAN

1.1 Latar Belakang

Data sekuensial memiliki urutan yang memengaruhi makna setiap nilai, seperti pada teks, suara, data sensor, dan deret waktu. Prediksi pada data semacam ini membutuhkan model yang membawa informasi dari langkah sebelumnya ke langkah berikutnya. Recurrent Neural Network atau RNN memenuhi kebutuhan tersebut melalui hidden state yang diteruskan antarlangkah [1], [2]. Pada urutan yang panjang, RNN dasar sulit mempelajari hubungan antarnilai yang berjauhan karena gradien mengecil ketika dirambatkan mundur, kondisi yang dikenal sebagai vanishing gradient [3].

Long Short-Term Memory atau LSTM dikembangkan untuk mengatasi keterbatasan tersebut dengan menambahkan cell state dan tiga gerbang yang mengatur aliran informasi [4]. Praktikum ini menerapkan LSTM dengan PyTorch untuk memprediksi deret sinus, kemudian membandingkannya dengan RNN dasar yang dilatih dengan konfigurasi yang sama. Hidden size dan jumlah layer diubah untuk melihat pengaruhnya terhadap MSE testing dan waktu pelatihan. Tugas individu menguji LSTM pada deret cosinus dengan tiga konfigurasi.

1.2 Rumusan Masalah

1. Bagaimana membentuk data sinus menjadi sequence dan membaginya menjadi data training dan testing?
2. Bagaimana membangun dan melatih model LSTM dan RNN menggunakan PyTorch untuk memprediksi nilai berikutnya?
3. Bagaimana perbandingan MSE testing dan kestabilan training antara LSTM dan RNN?

4. Bagaimana pengaruh hidden size dan jumlah layer terhadap MSE testing dan waktu training LSTM?
5. Apa peran cell state dan gerbang pada LSTM dalam menangani masalah vanishing gradient?

1.3 Tujuan Praktikum

1. Menjelaskan arsitektur dan cara kerja LSTM, termasuk cell state dan tiga gerbangnya.
2. Mengimplementasikan model LSTM dan RNN menggunakan PyTorch.
3. Melatih model untuk memprediksi deret sinus dan cosinus.
4. Mengevaluasi model menggunakan MSE testing, bentuk kurva prediksi, dan waktu training.
5. Menganalisis pengaruh hidden size dan jumlah layer serta membandingkan LSTM dengan RNN.

1.4 Manfaat Praktikum

Praktikum ini membantu memahami cara kerja LSTM melalui data yang sederhana dan mudah divisualisasikan, sehingga perbedaannya dengan RNN dapat diamati langsung pada kurva loss dan kurva prediksi. Hasil eksperimen juga menunjukkan bahwa hidden size atau jumlah layer yang lebih besar tidak otomatis memberi hasil yang lebih baik, dan keduanya menambah waktu komputasi.

BAB II

LANDASAN TEORI

2.1 Data Sekuensial dan Prediksi Deret Waktu

Deret waktu adalah data sekuensial yang nilainya dicatat menurut urutan waktu. Pada prediksi satu langkah ke depan, sejumlah nilai sebelumnya dipakai sebagai masukan untuk memperkirakan nilai berikutnya. Karena nilai yang diprediksi berupa angka kontinu, tugas ini termasuk regresi sehingga kesalahannya diukur dari selisih nilai, bukan dari akurasi kelas [1].

2.2 Recurrent Neural Network

RNN memproses data satu langkah demi satu langkah. Pada tiap langkah, hidden state dihitung dari masukan saat ini dan hidden state langkah sebelumnya, sehingga jaringan membawa ringkasan nilai yang telah dilihat [2]. Saat pelatihan, gradien dirambatkan mundur melalui seluruh langkah waktu. Perkalian yang berulang membuat gradien dapat mengecil atau membesar pada urutan panjang, sehingga RNN dasar kesulitan mempelajari hubungan yang jaraknya jauh [3].

2.3 Long Short-Term Memory

LSTM adalah jenis RNN yang menambahkan cell state sebagai jalur memori. Cell state membawa informasi dari satu langkah ke langkah berikutnya dengan perubahan yang kecil, dan perubahan itu diatur oleh gerbang [4]. Susunan ini memberi gradien jalur yang lebih stabil saat backpropagation, sehingga LSTM lebih mampu menangkap ketergantungan jangka panjang dibanding RNN dasar. Sebagai konsekuensinya, setiap layer LSTM memiliki empat kelompok bobot sehingga jumlah parameternya sekitar empat kali RNN dengan hidden size yang sama.

2.4 Gerbang pada LSTM

LSTM memiliki tiga gerbang. Forget gate (f_t) menentukan bagian cell state sebelumnya yang dilupakan [5]. Input gate (i_t) menentukan informasi baru yang disimpan ke cell state. Output gate (o_t) menentukan bagian cell state yang dikeluarkan sebagai hidden state. Perhitungannya ditunjukkan pada persamaan berikut.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f) \quad (2.1)$$

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i) \quad (2.2)$$

$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C) \quad (2.3)$$

$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t \quad (2.4)$$

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o) \quad (2.5)$$

$$h_t = o_t * \tanh(C_t) \quad (2.6)$$

Fungsi sigmoid (σ) menghasilkan nilai antara 0 dan 1 yang berperan sebagai pengatur seberapa banyak informasi diloloskan, sedangkan tanh menghasilkan kandidat informasi baru $\tilde{C}_t$.

2.5 Pembentukan Sequence dan Pembagian Data

Deret sepanjang 1.000 nilai diubah menjadi pasangan masukan dan target dengan sliding window. Sequence length ditetapkan sebesar 20, sehingga setiap sampel berisi dua puluh nilai berurutan dan targetnya adalah nilai ke-21. Proses ini menghasilkan 980 pasangan data. Pasangan data dibagi berdasarkan urutan, 80 persen untuk training dan 20 persen untuk testing, tanpa diacak agar urutan waktu tetap terjaga dan data uji berasal dari bagian akhir deret.

2.6 Fungsi Loss dan Optimizer Adam

Mean Squared Error (MSE) dipakai sebagai fungsi loss dan ukuran evaluasi karena tugas yang dikerjakan berupa regresi. MSE menghitung rata-rata kuadrat selisih antara nilai prediksi dan target, sehingga nilai yang lebih kecil menunjukkan prediksi yang lebih dekat dengan data asli. Bobot diperbarui dengan optimizer Adam dengan

learning rate 0,01 [6]. Adam memperkirakan momen pertama dan kedua dari gradien sehingga laju pembaruan dapat menyesuaikan setiap parameter. Pelatihan dijalankan selama 50 epoch, dan setiap epoch memproses seluruh data training sekaligus.

2.7 Hidden Size dan Jumlah Layer

Hidden size menentukan jumlah neuron pada hidden state, sedangkan jumlah layer menentukan banyaknya lapisan LSTM yang ditumpuk. Nilai yang lebih besar memberi kapasitas representasi yang lebih luas, tetapi menambah jumlah parameter dan beban komputasi. Kapasitas yang lebih besar belum tentu menurunkan error karena hasilnya juga dipengaruhi inisialisasi bobot, learning rate, jumlah epoch, dan karakter data.

2.8 Deret Sinus dan Cosinus

Fungsi sinus dan cosinus memiliki pola periodik yang teratur dan bentuk yang sama dengan fase yang berbeda. Keduanya cocok untuk latihan awal karena keberhasilan model dapat dilihat langsung dari kesesuaian kurva prediksi dengan kurva asli.

BAB III

METODOLOGI DAN LANGKAH PRAKTIKUM

3.1 Alat dan Bahan

Alat dan bahan yang digunakan dalam praktikum ini adalah sebagai berikut:

1. Laptop sebagai perangkat untuk menjalankan notebook.
2. Google Colaboratory dengan GPU sebagai lingkungan pemrograman.
3. Python sebagai bahasa pemrograman.
4. PyTorch untuk membangun dan melatih model RNN dan LSTM [7].
5. NumPy untuk membentuk data numerik.
6. Matplotlib untuk menampilkan grafik.
7. Pandas untuk menyusun tabel hasil eksperimen.
8. Notebook 2411531001_Habibi_Putra_Rizqullah_Program4.ipynb sebagai berkas kerja.

3.2 Alur Pengerjaan

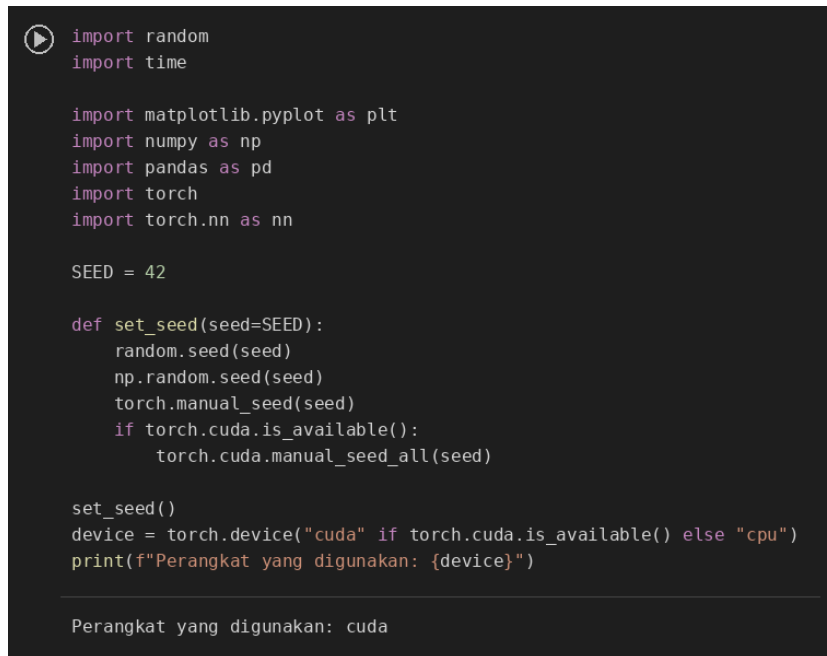
Pengerjaan dimulai dengan menetapkan seed dan memilih perangkat komputasi. Data sinus dibuat sebanyak 1.000 titik, diubah menjadi sequence sepanjang 20, lalu dibagi menjadi 784 sampel training dan 196 sampel testing. RNN dan LSTM dilatih dengan konfigurasi yang sama dan dibandingkan, kemudian LSTM dilatih ulang dengan hidden size 32 dan 128. Pada tugas praktikum, pola data diganti menjadi cosinus dan tiga konfigurasi LSTM diuji, yaitu hidden size 32 dengan satu layer, hidden size 64 dengan satu layer, dan hidden size 64 dengan dua layer. Urutan langkah mengikuti modul praktikum [8].

3.3 Praktikum Prediksi Deret Sinus

3.3.1 Persiapan Library dan Perangkat

Library random dan NumPy dipakai untuk membangkitkan data dan mengatur seed, sedangkan PyTorch dipakai untuk membentuk model. Seed 42 ditetapkan melalui

fungsi `set_seed`, yang dipanggil kembali sebelum setiap eksperimen supaya inisialisasi bobot sama dan perbandingan antarmodel lebih adil. Perangkat dipilih secara otomatis dan pada pelaksanaan ini model berjalan menggunakan GPU (CUDA).



```
import random
import time

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn

SEED = 42

def set_seed(seed=SEED):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

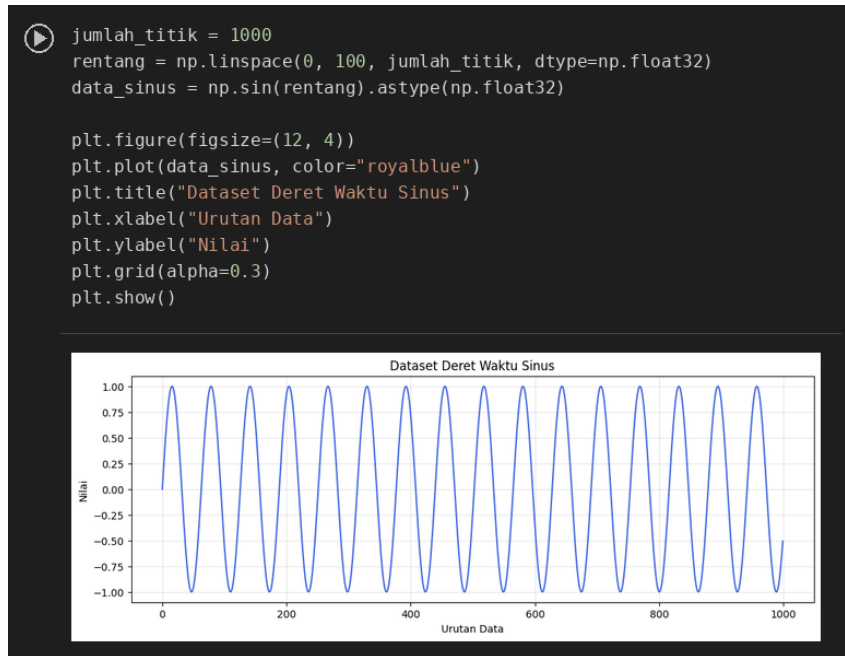
set_seed()
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Perangkat yang digunakan: {device}")

Perangkat yang digunakan: cuda
```

Gambar 3.1 Import library dan penentuan perangkat

3.3.2 Pembuatan Data Sinus

Data dibuat dengan fungsi sinus pada rentang 0 sampai 100 sebanyak 1.000 titik. Grafik menunjukkan pola periodik dengan nilai antara -1 dan 1, sehingga data siap dipakai untuk pembentukan sequence.



Gambar 3.2 Visualisasi data deret sinus

3.3.3 Pembentukan Sequence dan Pembagian Data

Fungsi `create_sequences` mengambil dua puluh nilai berurutan sebagai masukan dan satu nilai berikutnya sebagai target, lalu menambahkan dimensi fitur sehingga bentuk masukan menjadi jumlah sampel x 20 x 1. Fungsi `prepare_data` membagi hasilnya secara berurutan dengan rasio 80 : 20. Hasilnya adalah 784 sampel training dan 196 sampel testing dari total 980 pasangan data.

```

▶ def create_sequences(data, seq_length=20):
    xs, ys = [], []
    for i in range(len(data) - seq_length):
        xs.append(data[i:i + seq_length])
        ys.append(data[i + seq_length])

    X = np.asarray(xs, dtype=np.float32)[..., np.newaxis]
    y = np.asarray(ys, dtype=np.float32)[..., np.newaxis]
    return X, y

def prepare_data(data, seq_length=20, train_ratio=0.8):
    X, y = create_sequences(data, seq_length)
    batas_train = int(len(X) * train_ratio)

    X_train = torch.from_numpy(X[:batas_train]).to(device)
    y_train = torch.from_numpy(y[:batas_train]).to(device)
    X_test = torch.from_numpy(X[batas_train:]).to(device)
    y_test = torch.from_numpy(y[batas_train:]).to(device)
    return X_train, y_train, X_test, y_test, batas_train

SEQ_LENGTH = 20
X_train, y_train, X_test, y_test, batas_train = prepare_data(
    data_sinus, seq_length=SEQ_LENGTH
)

print(f"Bentuk X training : {tuple(X_train.shape)}")
print(f"Bentuk y training : {tuple(y_train.shape)}")
print(f"Bentuk X testing : {tuple(X_test.shape)}")
print(f"Bentuk y testing : {tuple(y_test.shape)}")

Bentuk X training : (784, 20, 1)
Bentuk y training : (784, 1)
Bentuk X testing : (196, 20, 1)
Bentuk y testing : (196, 1)

```

Gambar 3.3 Pembentukan sequence dan pembagian data sinus

3.3.4 Definisi Model RNN dan LSTM

RNNModel dan LSTMModel memiliki susunan yang sejajar, yaitu satu lapisan recurrent yang diikuti satu lapisan linear. Hidden size dan jumlah layer dibuat sebagai parameter agar mudah diubah pada eksperimen. Keluaran pada langkah waktu terakhir diteruskan ke lapisan linear karena bagian itu telah membawa informasi dari seluruh dua puluh nilai masukan. Dengan hidden size 64 dan satu layer, RNN memiliki 4.353 parameter dan LSTM memiliki 17.217 parameter.

```

class RNNModel(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, output_size=1, num_layers=1):
        super().__init__()
        self.rnn = nn.RNN(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
        )
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out, _ = self.rnn(x)
        return self.fc(out[:, -1, :])

class LSTMModel(nn.Module):
    def __init__(self, input_size=1, hidden_size=64, output_size=1, num_layers=1):
        super().__init__()
        self.lstm = nn.LSTM(
            input_size=input_size,
            hidden_size=hidden_size,
            num_layers=num_layers,
            batch_first=True,
        )
        self.fc = nn.Linear(hidden_size, output_size)

    def forward(self, x):
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])

```

Gambar 3.4 Definisi model RNN dan LSTM

3.3.5 Fungsi Pelatihan dan Evaluasi

Fungsi `train_model` dibuat agar pelatihan dapat dipakai ulang pada semua eksperimen. Fungsi menerima model, data training, jumlah epoch, dan learning rate. Setiap epoch menjalankan forward pass, menghitung MSE, melakukan backpropagation, dan memperbarui bobot dengan Adam, sementara waktu pelatihan dicatat dengan `time.perf_counter`.

```

def train_model(model, X_train, y_train, epochs=50, learning_rate=0.01, verbose=True):
    model = model.to(device)
    criterion = nn.MSELoss()
    optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
    history = []

    start_time = time.perf_counter()
    for epoch in range(epochs):
        model.train()
        optimizer.zero_grad()

        predictions = model(X_train)
        loss = criterion(predictions, y_train)
        loss.backward()
        optimizer.step()

        history.append(loss.item())
        if verbose and (epoch + 1) % 10 == 0:
            print(f"Epoch [{epoch + 1:02d}/{epochs}] | Loss: {loss.item():.6f}")

    training_time = time.perf_counter() - start_time
    return history, training_time

```

Gambar 3.5 Fungsi pelatihan model

Fungsi `evaluate_model` memindahkan model ke mode evaluasi dan menghitung MSE pada data testing tanpa gradien. Prediksi dikembalikan dalam bentuk array NumPy agar dapat digambar bersama data asli.

```

def evaluate_model(model, X_test, y_test):
    model.eval()
    criterion = nn.MSELoss()
    with torch.no_grad():
        predictions = model(X_test)
        mse = criterion(predictions, y_test).item()
    return predictions.detach().cpu().numpy().ravel(), mse

```

Gambar 3.6 Fungsi evaluasi model

3.3.6 Pelatihan RNN dan LSTM Dasar

RNN dan LSTM dilatih dengan hidden size 64, satu layer, 50 epoch, dan learning rate 0,01. Loss RNN turun dari 0,031718 pada epoch ke-10 menjadi 0,000073 pada epoch ke-50, sedangkan loss LSTM turun dari 0,033891 menjadi 0,000427. Pada data testing, RNN memperoleh MSE 0,000364 dengan waktu 1,35 detik dan LSTM memperoleh MSE 0,000282 dengan waktu 0,16 detik.

```
▶ EPOCHS = 50
  LEARNING_RATE = 0.01

set_seed()
rnn_model = RNNModel(hidden_size=64, num_layers=1)
print("Training RNN")
rnn_losses, rnn_time = train_model(
    rnn_model, X_train, y_train, EPOCHS, LEARNING_RATE
)
rnn_predictions, rnn_mse = evaluate_model(rnn_model, X_test, y_test)

set_seed()
lstm_model = LSTMModel(hidden_size=64, num_layers=1)
print("\nTraining LSTM")
lstm_losses, lstm_time = train_model(
    lstm_model, X_train, y_train, EPOCHS, LEARNING_RATE
)
lstm_predictions, lstm_mse = evaluate_model(lstm_model, X_test, y_test)

print("\nHasil pengujian")
print(f"RNN | MSE testing: {rnn_mse:.6f} | Waktu training: {rnn_time:.2f} detik")
print(f"LSTM | MSE testing: {lstm_mse:.6f} | Waktu training: {lstm_time:.2f} detik")

Training RNN
Epoch [10/50] | Loss: 0.031718
Epoch [20/50] | Loss: 0.017357
Epoch [30/50] | Loss: 0.002829
Epoch [40/50] | Loss: 0.001083
Epoch [50/50] | Loss: 0.000073

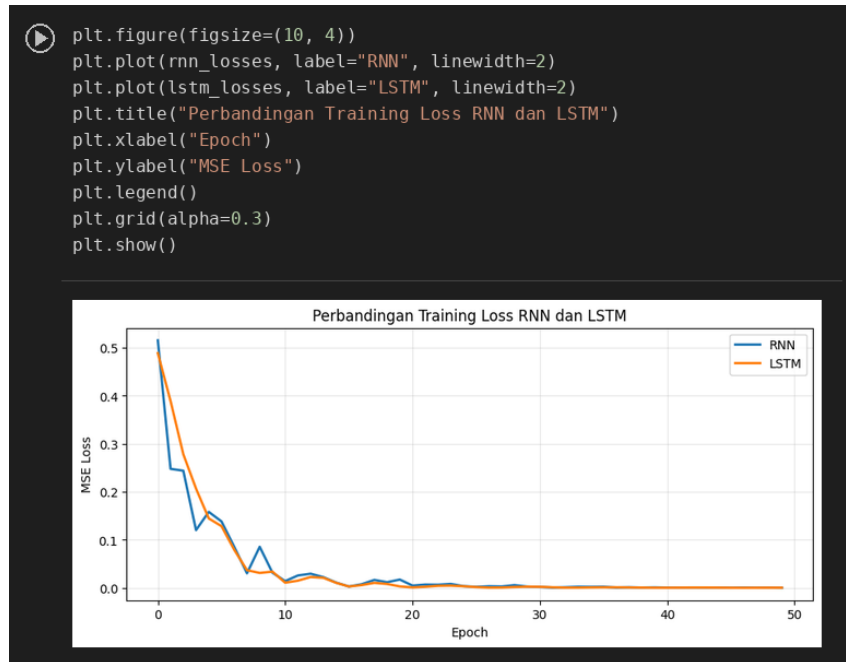
Training LSTM
Epoch [10/50] | Loss: 0.033891
Epoch [20/50] | Loss: 0.003125
Epoch [30/50] | Loss: 0.002594
Epoch [40/50] | Loss: 0.000390
Epoch [50/50] | Loss: 0.000427

Hasil pengujian
RNN | MSE testing: 0.000364 | Waktu training: 1.35 detik
LSTM | MSE testing: 0.000282 | Waktu training: 0.16 detik
```

Gambar 3.7 Hasil pelatihan RNN dan LSTM dasar

3.3.7 Visualisasi Training Loss

Kedua kurva turun tajam pada sepuluh epoch pertama dan mendekati nol setelah sekitar epoch ke-30. Kurva RNN lebih bergerigi, dengan kenaikan loss sementara di sekitar epoch ke-4 dan ke-8, sedangkan kurva LSTM turun lebih mulus.



Gambar 3.8 Grafik perbandingan training loss RNN dan LSTM

3.3.8 Prediksi Data Testing Sinus

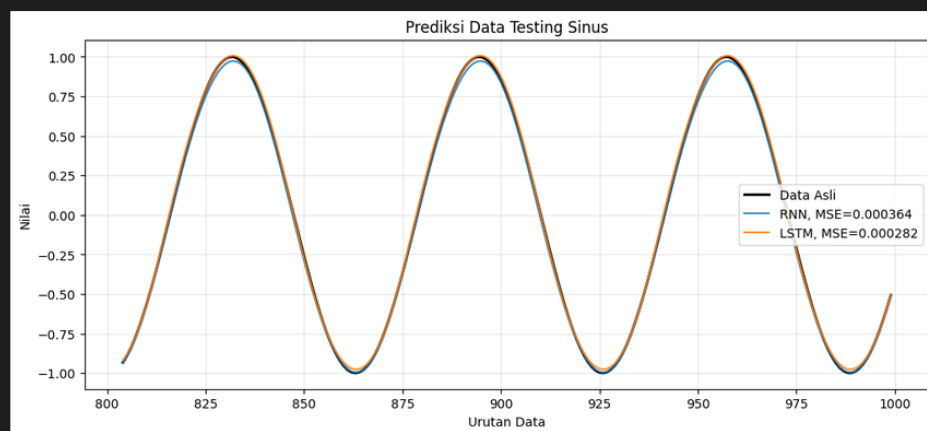
Prediksi kedua model dibandingkan dengan data asli pada indeks 804 sampai 999, yang mencakup tiga periode gelombang. Keduanya mengikuti bentuk gelombang dengan baik. Prediksi RNN sedikit lebih rendah dari data asli di puncak gelombang, sedangkan prediksi LSTM lebih dekat dengan garis data asli.

```

y_test_numpy = y_test.detach().cpu().numpy().ravel()
indeks_test = np.arange(batas_train + SEQ_LENGTH, len(data_sinus))

plt.figure(figsize=(12, 5))
plt.plot(indeks_test, y_test_numpy, label="Data Asli", color="black", linewidth=2)
plt.plot(indeks_test, rnn_predictions, label=f"RNN, MSE={rnn_mse:.6f}", alpha=0.85)
plt.plot(indeks_test, lstm_predictions, label=f"LSTM, MSE={lstm_mse:.6f}", alpha=0.85)
plt.title("Prediksi Data Testing Sinus")
plt.xlabel("Urutan Data")
plt.ylabel("Nilai")
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```



Gambar 3.9 Perbandingan data asli dan prediksi pada data testing sinus

3.3.9 Eksperimen Hidden Size

Modul meminta percobaan hidden size 32, 64, dan 128. Hasil LSTM dengan hidden size 64 dari langkah sebelumnya dipakai kembali, sedangkan hidden size 32 dan 128 dilatih dengan seed, epoch, dan learning rate yang sama. Hasil setiap model disimpan dalam DataFrame berisi loss akhir training, MSE testing, dan waktu training.

```

hasil_hidden = []
model_hidden = {}

for hidden_size in [32, 64, 128]:
    if hidden_size == 64:
        model_eksperimen = lstm_model
        losses = lstm_losses
        waktu = lstm_time
        predictions = lstm_predictions
        mse = lstm_mse
    else:
        set_seed()
        model_eksperimen = LSTMModel(hidden_size=hidden_size, num_layers=1)
        print(f"Training LSTM dengan hidden size {hidden_size}")
        losses, waktu = train_model(
            model_eksperimen,
            X_train,
            y_train,
            epochs=EPOCHS,
            learning_rate=LEARNING_RATE,
            verbose=False,
        )
        predictions, mse = evaluate_model(model_eksperimen, X_test, y_test)

    model_hidden[hidden_size] = {
        "model": model_eksperimen,
        "losses": losses,
        "predictions": predictions,
    }
    hasil_hidden.append({
        "Hidden Size": hidden_size,
        "Num Layers": 1,
        "Loss Akhir Training": losses[-1],
        "MSE Testing": mse,
        "Waktu Training (s)": waktu,
    })

tabel_hidden = pd.DataFrame(hasil_hidden).round(6)
tabel_hidden

```

Gambar 3.10 Kode eksperimen hidden size

MSE testing turun dari 0,000647 pada hidden size 32 menjadi 0,000211 pada hidden size 128, sedangkan waktu training naik dari 0,137526 detik menjadi 0,418476 detik.

```

*** Training LSTM dengan hidden size 32
*** Training LSTM dengan hidden size 128

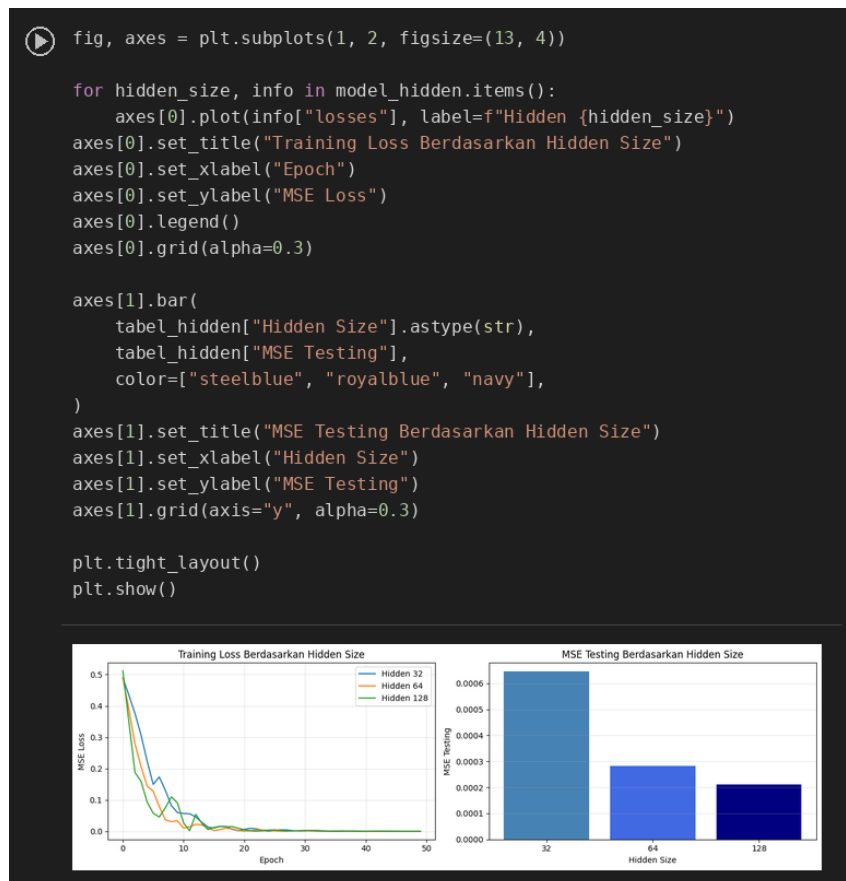
```

	Hidden Size	Num Layers	Loss Akhir Training	MSE Testing	Waktu Training (s)
0	32	1	0.000529	0.000647	0.137526
1	64	1	0.000427	0.000282	0.156210
2	128	1	0.000191	0.000211	0.418476

Gambar 3.11 Tabel hasil eksperimen hidden size

3.3.10 Visualisasi Eksperimen Hidden Size

Grafik pertama membandingkan kurva loss setiap hidden size, dan grafik kedua membandingkan MSE testing. Hidden size 32 turun paling lambat, sedangkan hidden size 64 dan 128 turun lebih cepat pada sepuluh epoch pertama. Hidden size 128 sempat mengalami kenaikan loss di sekitar epoch ke-8 dan ke-12 sebelum turun kembali.



Gambar 3.12 Grafik loss dan MSE testing untuk setiap hidden size

3.3.11 Ringkasan Hasil Praktikum

Program menyusun ringkasan dari hasil yang diperoleh. LSTM memiliki MSE testing lebih rendah daripada RNN dengan selisih 0,000082, dan hidden size 128 menjadi yang terbaik dengan MSE testing 0,000211. Ringkasan juga mencatat bahwa

kurva yang tampak halus belum tentu memiliki kesalahan terkecil, sehingga kurva prediksi perlu dibaca bersama nilai MSE.

```
▶ model_terbaik_hidden = tabel_hidden.loc[tabel_hidden["MSE Testing"].idxmin()]

print("Analisis hasil praktikum")
if lstm_mse < rnn_mse:
    print(f"1. LSTM memperoleh MSE testing lebih rendah daripada RNN dengan selisih {rnn_mse - lstm_mse:.6f}.")
elif lstm_mse > rnn_mse:
    print(f"1. Pada data sederhana ini, RNN memperoleh MSE testing lebih rendah daripada LSTM dengan selisih {lstm_mse - rnn_mse:.6f}.")
else:
    print("1. RNN dan LSTM memperoleh MSE testing yang sama.")

print(
    f"2. Hidden size terbaik pada percobaan ini adalah {int(model_terbaik_hidden['Hidden Size'])} "
    f"dengan MSE testing {model_terbaik_hidden['MSE Testing']:.6f}."
)
print(
    "3. Penambahan hidden size meningkatkan kapasitas model, tetapi hasilnya tidak selalu lebih baik "
    "dan waktu training cenderung bertambah."
)
print(
    "4. Kurva prediksi perlu dilihat bersama nilai MSE karena kurva yang tampak halus belum tentu "
    "memiliki kesalahan paling kecil."
)

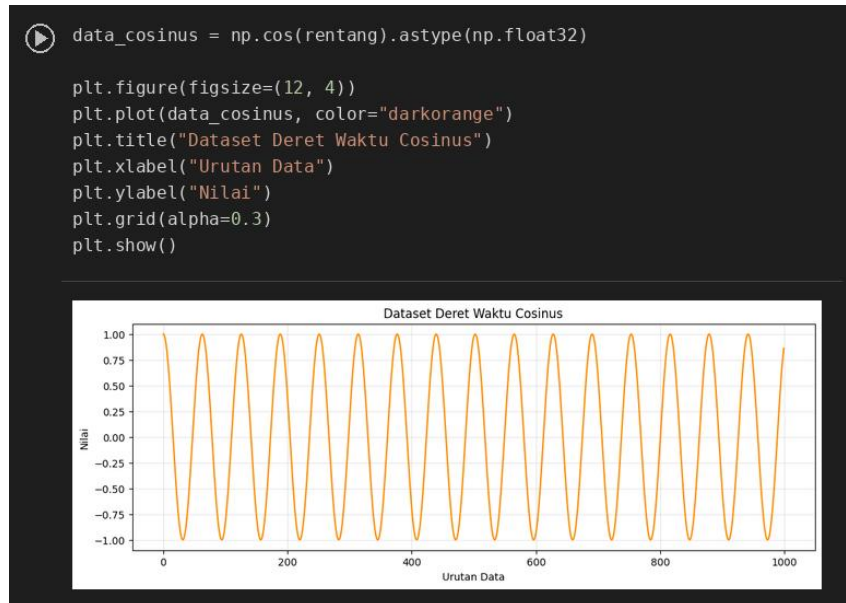
Analisis hasil praktikum
1. LSTM memperoleh MSE testing lebih rendah daripada RNN dengan selisih 0.000082.
2. Hidden size terbaik pada percobaan ini adalah 128 dengan MSE testing 0.000211.
3. Penambahan hidden size meningkatkan kapasitas model, tetapi hasilnya tidak selalu lebih baik dan waktu training cenderung bertambah.
4. Kurva prediksi perlu dilihat bersama nilai MSE karena kurva yang tampak halus belum tentu memiliki kesalahan paling kecil.
```

Gambar 3.13 Ringkasan hasil praktikum

3.4 Tugas Praktikum Prediksi Deret Cosinus

3.4.1 Pembuatan Data Cosinus

Data tugas dibuat dengan fungsi cosinus pada rentang dan jumlah titik yang sama dengan praktikum. Hanya fungsi pembangkitnya yang berubah, sehingga perbandingan antarkonfigurasi tetap sama.



Gambar 3.14 Visualisasi data deret cosinus

3.4.2 Pembentukan Sequence Data Cosinus

Fungsi `prepare_data` yang sama dipakai untuk data cosinus dengan `sequence length` 20. Hasilnya adalah 784 sampel training dan 196 sampel testing, sama seperti pada data sinus.

```
X_train_cos, y_train_cos, X_test_cos, y_test_cos, batas_train_cos = prepare_data(
    data_cosinus, seq_length=SEQ_LENGTH
)

print(f"Jumlah sequence training: {len(X_train_cos)}")
print(f"Jumlah sequence testing : {len(X_test_cos)}")

Jumlah sequence training: 784
Jumlah sequence testing : 196
```

Gambar 3.15 Pembentukan sequence data cosinus

3.4.3 Konfigurasi dan Pelatihan Tiga Model LSTM

Tiga konfigurasi didaftarkan dalam sebuah list. LSTM_1 memakai hidden size 32 dan satu layer, LSTM_2 memakai hidden size 64 dan satu layer, dan LSTM_3 memakai hidden size 64 dan dua layer sesuai instruksi tugas. Seed diatur ulang sebelum setiap model dibuat, lalu ketiganya dilatih dengan 50 epoch dan learning rate 0,01.

```

▶ konfigurasi_tugas = [
    {"Model": "LSTM_1", "Hidden Size": 32, "Num Layers": 1},
    {"Model": "LSTM_2", "Hidden Size": 64, "Num Layers": 1},
    {"Model": "LSTM_3", "Hidden Size": 64, "Num Layers": 2},
]

hasil_tugas = []
detail_tugas = {}

for config in konfigurasi_tugas:
    set_seed()
    model_tugas = LSTMModel(
        hidden_size=config["Hidden Size"],
        num_layers=config["Num Layers"],
    )

    print(
        f"Training {config['Model']} | hidden={config['Hidden Size']} | "
        f"layers={config['Num Layers']}"
    )
    losses, waktu = train_model(
        model_tugas,
        X_train_cos,
        y_train_cos,
        epochs=EPOCHS,
        learning_rate=LEARNING_RATE,
        verbose=False,
    )
    predictions, mse = evaluate_model(
        model_tugas, X_test_cos, y_test_cos
    )

```

Gambar 3.16 Konfigurasi dan pelatihan tiga model LSTM

3.4.4 Penyusunan Tabel Hasil

Setelah pelatihan, setiap model dievaluasi pada data testing. Loss akhir, MSE testing, waktu training, dan catatan disimpan dalam list lalu diubah menjadi DataFrame. Nilai numerik dibulatkan sampai enam angka di belakang koma agar perbedaan kecil tetap terlihat.

```

catatan = (
    "Model lebih ringan"
    if config["Hidden Size"] == 32
    else "Kapasitas hidden lebih besar"
    if config["Num Layers"] == 1
    else "Menggunakan dua layer LSTM"
)

hasil_tugas.append({
    "Model": config["Model"],
    "Hidden Size": config["Hidden Size"],
    "Num Layers": config["Num Layers"],
    "Loss Akhir": losses[-1],
    "MSE Testing": mse,
    "Waktu Training (s)": waktu,
    "Catatan": catatan,
})

detail_tugas[config["Model"]] = {
    "model": model_tugas,
    "losses": losses,
    "predictions": predictions,
}

tabel_tugas = pd.DataFrame(hasil_tugas)
tabel_tugas_tampil = tabel_tugas.copy()
tabel_tugas_tampil[["Loss Akhir", "MSE Testing", "Waktu Training (s)"]] = (
    tabel_tugas_tampil[["Loss Akhir", "MSE Testing", "Waktu Training (s)"]].round(6)
)
tabel_tugas_tampil

```

Gambar 3.17 Penyimpanan hasil eksperimen ke DataFrame

Tabel menunjukkan LSTM_2 memiliki MSE testing terendah, yaitu 0,000205, sedangkan LSTM_3 memiliki MSE testing tertinggi, yaitu 0,001253.

```

*** Training LSTM_1 | hidden=32 | layers=1
Training LSTM_2 | hidden=64 | layers=1
Training LSTM_3 | hidden=64 | layers=2

```

	Model	Hidden Size	Num Layers	Loss Akhir	MSE Testing	Waktu Training (s)	Catatan
0	LSTM_1	32	1	0.000452	0.000577	0.106139	Model lebih ringan
1	LSTM_2	64	1	0.000255	0.000205	0.111731	Kapasitas hidden lebih besar
2	LSTM_3	64	2	0.001538	0.001253	0.167531	Menggunakan dua layer LSTM

Gambar 3.18 Tabel hasil eksperimen data cosinus

3.4.5 Perbandingan Training Loss dan MSE Testing

Kurva loss ketiga model digambar dalam satu grafik, dan MSE testing ditampilkan sebagai diagram batang. Kurva LSTM_3 melonjak kembali ke sekitar 0,5

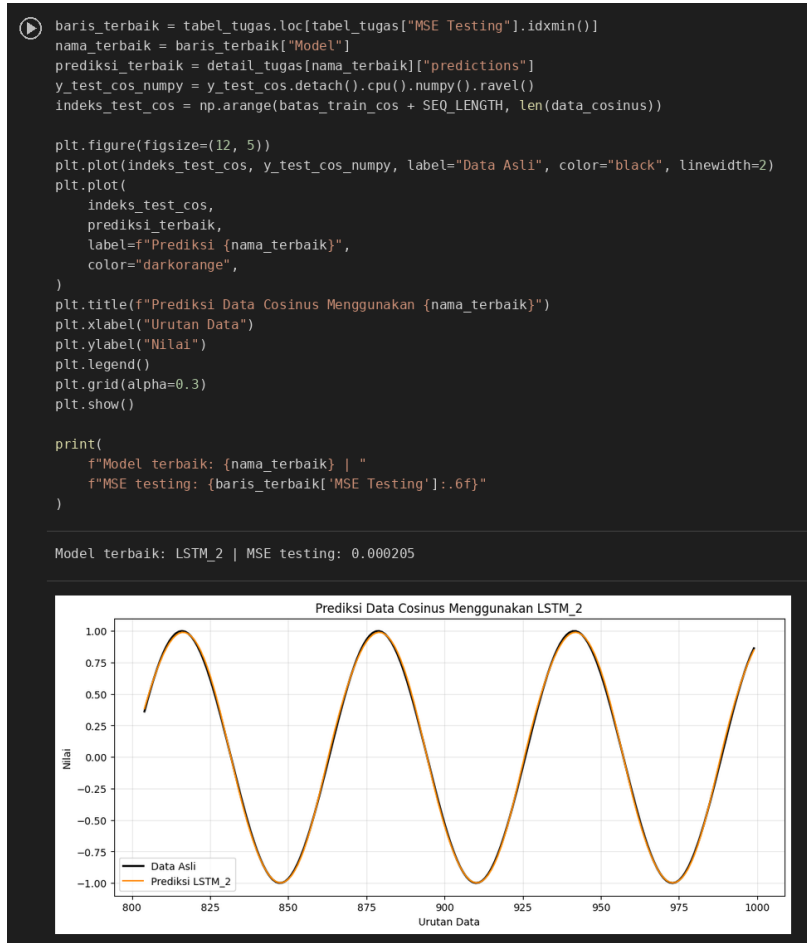
pada epoch ke-3 dan bertahan di kisaran 0,2 sampai 0,27 hingga sekitar epoch ke-8, sedangkan kurva LSTM_1 dan LSTM_2 turun lebih teratur.



Gambar 3.19 Grafik loss dan MSE testing pada data cosinus

3.4.6 Prediksi Model Terbaik

Program memilih model dengan MSE testing terendah secara otomatis, yaitu LSTM_2, lalu membandingkan prediksinya dengan data asli. Garis prediksi hampir berimpit dengan garis data asli pada seluruh bagian data testing.



Gambar 3.20 Prediksi model terbaik pada data cosinus

3.4.7 Analisis Pengaruh Penambahan Layer

LSTM_2 dan LSTM_3 dibandingkan karena memakai hidden size yang sama, sehingga selisih hasilnya berasal dari jumlah layer. Program mencatat bahwa dua layer menaikkan MSE testing sebesar 0,001048 dan menambah waktu training sebesar 0,06 detik.

```

lstm_2 = tabel_tugas[tabel_tugas["Model"] == "LSTM_2"].iloc[0]
lstm_3 = tabel_tugas[tabel_tugas["Model"] == "LSTM_3"].iloc[0]

selisih_mse = lstm_3["MSE Testing"] - lstm_2["MSE Testing"]
selisih_waktu = lstm_3["Waktu Training (s)"] - lstm_2["Waktu Training (s)"]

print("Analisis penambahan layer")
if selisih_mse < 0:
    print(f"1. Dua layer menurunkan MSE testing sebesar {abs(selisih_mse):.6f}.")
elif selisih_mse > 0:
    print(f"1. Dua layer menaikkan MSE testing sebesar {selisih_mse:.6f}.")
else:
    print("1. Satu layer dan dua layer menghasilkan MSE testing yang sama.")

if selisih_waktu > 0:
    print(f"2. LSTM dua layer membutuhkan waktu training {selisih_waktu:.2f} detik lebih lama.")
else:
    print(f"2. Pada proses ini, LSTM dua layer selesai {abs(selisih_waktu):.2f} detik lebih cepat.")

print(
    "3. Penambahan layer meningkatkan kedalaman model dan jumlah komputasi. "
    "Namun, model yang lebih dalam tidak selalu memberikan MSE yang lebih rendah pada data sederhana "
    "."
)

Analisis penambahan layer
1. Dua layer menaikkan MSE testing sebesar 0.001048.
2. LSTM dua layer membutuhkan waktu training 0.06 detik lebih lama.
3. Penambahan layer meningkatkan kedalaman model dan jumlah komputasi. Namun, model yang lebih dalam tidak s
    elalu memberikan MSE yang lebih rendah pada data sederhana.

```

Gambar 3.21 Analisis pengaruh penambahan layer

BAB IV

HASIL DAN PEMBAHASAN

4.1 Hasil Praktikum Deret Sinus

4.1.1 Hasil RNN dan LSTM Dasar

Model	Hidden Size	Num Layers	Loss Akhir Training	MSE Testing	Waktu Training (s)	Catatan
RNN	64	1	0,000073	0,000364	1,35	Loss training terendah
LSTM	64	1	0,000427	0,000282	0,16	MSE testing lebih rendah

Tabel 4.1 Hasil perbandingan RNN dan LSTM pada data sinus

LSTM memperoleh MSE testing 0,000282, lebih rendah daripada RNN yang sebesar 0,000364, dengan selisih 0,000082. Selisih ini kecil dan berasal dari satu kali percobaan dengan satu seed, sehingga belum cukup kuat untuk menyatakan LSTM unggul pada data sinus. Loss training pada epoch terakhir justru lebih rendah pada RNN (0,000073) daripada LSTM (0,000427). Nilai itu hanya mewakili epoch ke-50, dan dengan satu percobaan perbedaan ini belum cukup untuk menyimpulkan bahwa RNN mengalami overfitting.

Waktu training RNN (1,35 detik) jauh lebih lama daripada LSTM (0,16 detik). Angka ini tidak mencerminkan kecepatan arsitektur, karena LSTM memiliki parameter sekitar empat kali lebih banyak. RNN dilatih lebih dulu pada sel yang sama sehingga waktunya kemungkinan mencakup inisialisasi awal GPU. Perbandingan waktu yang adil memerlukan pengukuran berulang dengan urutan pelatihan yang dibalik.

4.1.2 Perbandingan Hidden Size

Hidden Size	Num Layers	Loss Akhir Training	MSE Testing	Waktu Training (s)	Catatan
32	1	0,000529	0,000647	0,137526	Paling ringan, MSE testing tertinggi

Hidden Size	Num Layers	Loss Akhir Training	MSE Testing	Waktu Training (s)	Catatan
64	1	0,000427	0,000282	0,156210	Dasar LSTM
128	1	0,000191	0,000211	0,418476	MSE testing terendah, paling lama

Tabel 4.2 Hasil eksperimen hidden size pada LSTM satu layer (data sinus)

MSE testing turun dari 0,000647 pada hidden size 32 menjadi 0,000282 pada hidden size 64 dan 0,000211 pada hidden size 128. Penurunan dari 64 ke 128 sebesar 0,000071 jauh lebih kecil daripada penurunan dari 32 ke 64 sebesar 0,000365, sementara waktu training naik sekitar 2,7 kali lipat, dari 0,156210 menjadi 0,418476 detik. Hidden size 128 memberikan MSE terendah, sedangkan hidden size 64 menjadi pilihan yang lebih hemat waktu dengan MSE yang masih rendah.

4.1.3 Analisis Loss dan Prediksi

Penurunan loss menunjukkan bahwa kedua model berhasil mempelajari hubungan antara dua puluh nilai masukan dan nilai berikutnya. Prediksi yang hampir berimpit dengan data asli pada data testing menandakan pola periodik tertangkap pada bagian data yang tidak dipakai untuk pelatihan. Kurva loss LSTM yang lebih mulus menunjukkan proses optimasi yang lebih stabil. Pengujian ini berupa prediksi satu langkah dengan masukan berupa data asli, sedangkan kestabilan pada prediksi berantai, yaitu ketika hasil prediksi dipakai sebagai masukan berikutnya, tidak diuji dalam percobaan ini.

4.2 Hasil Tugas Praktikum Deret Cosinus

4.2.1 Hasil Eksperimen

Model	Hidden Size	Num Layers	Loss Akhir	MSE Testing	Waktu Training (s)	Catatan
LSTM_1	32	1	0,000452	0,000577	0,106139	MSE testing sekitar 2,8 kali LSTM_2

Model	Hidden Size	Num Layers	Loss Akhir	MSE Testing	Waktu Training (s)	Catatan
LSTM_2	64	1	0,000255	0,000205	0,111731	MSE testing terendah
LSTM_3	64	2	0,001538	0,001253	0,167531	MSE testing tertinggi, konvergen lambat

Tabel 4.3 Hasil eksperimen LSTM pada data cosinus

LSTM_2 menghasilkan MSE testing terendah, yaitu 0,000205. LSTM_1 dengan hidden size 32 memiliki MSE testing 0,000577, sekitar 2,8 kali LSTM_2, sejalan dengan data sinus bahwa hidden size yang terlalu kecil membatasi kemampuan model. Waktu training LSTM_1 dan LSTM_2 hampir sama (0,106139 dan 0,111731 detik), sehingga pada ukuran data ini penambahan hidden size dari 32 ke 64 tidak terasa pada waktu.

4.2.2 Pengaruh Penambahan Layer

LSTM_3 memakai hidden size yang sama dengan LSTM_2 tetapi dengan dua layer. MSE testing naik dari 0,000205 menjadi 0,001253, sekitar 6,1 kali lipat, dan loss akhir training naik dari 0,000255 menjadi 0,001538. Waktu training bertambah 0,06 detik, dari 0,111731 menjadi 0,167531 detik, dan jumlah parameter naik dari 17.217 menjadi 50.497. Tabel contoh pada modul menunjukkan LSTM dua layer dengan loss paling kecil, sedangkan pada percobaan ini hasilnya berlawanan.

Kurva loss memberi petunjuk penyebabnya. Loss LSTM_3 melonjak kembali ke sekitar 0,5 pada epoch ke-3, bertahan di kisaran 0,2 sampai 0,27 hingga sekitar epoch ke-8, dan baru mendekati nol setelah sekitar epoch ke-30. Dua layer tampaknya membutuhkan lebih banyak epoch atau learning rate yang lebih kecil dari 0,01 agar konvergen sebaik model satu layer. Dugaan ini belum diuji dan dapat dicoba dengan menambah jumlah epoch atau menurunkan learning rate.

4.3 Perbandingan LSTM dan RNN

LSTM sedikit lebih baik daripada RNN pada MSE testing dan memiliki kurva loss yang lebih mulus, tetapi keunggulannya kecil dan hanya diuji satu kali. RNN pada percobaan ini dilatih ulang dengan sequence length 20 dan data testing terpisah, sehingga hasilnya tidak dibandingkan langsung dengan laporan Minggu 3 yang memakai sequence length 10 dan seluruh 990 pasangan data.

Data sinus dan cosinus dengan jendela 20 langkah memiliki pola sederhana, dan RNN sudah mampu mencapai loss training 0,000073. Cell state dan gerbang pada LSTM baru benar-benar diperlukan pada urutan yang panjang atau pola dengan ketergantungan jangka panjang, yang tidak diwakili oleh data pada praktikum ini. Untuk memastikan perbedaannya, percobaan perlu diulang dengan beberapa seed, data cosinus, dan urutan masukan yang lebih panjang.

4.4 Jawaban Pertanyaan Reflektif

4.4.1 Peran Cell State dalam LSTM

Cell state berfungsi sebagai jalur memori utama yang membawa informasi dari satu langkah waktu ke langkah berikutnya dengan perubahan yang kecil. Isinya diperbarui oleh forget gate yang membuang informasi lama dan input gate yang menambahkan informasi baru. Hidden state yang menjadi keluaran pada setiap langkah diambil dari cell state melalui output gate [4].

4.4.2 Alasan LSTM Lebih Mampu Mengingat Konteks Jangka Panjang

Pembaruan cell state pada LSTM bersifat aditif dan diatur oleh gerbang, sehingga gradien tidak harus melewati perkalian bobot dan fungsi aktivasi yang berulang pada setiap langkah seperti pada RNN. Dengan jalur ini gradien tidak mudah hilang ketika dirambatkan mundur, sehingga vanishing gradient berkurang dan informasi dari langkah yang jauh masih dapat memengaruhi keluaran [3], [4]. Pada percobaan ini efek tersebut belum terlihat jelas karena jendela hanya 20 langkah.

4.4.3 Dampak Penambahan Jumlah Layer

Penambahan layer membuat model dapat mempelajari representasi yang lebih kompleks, tetapi parameter dan beban komputasinya bertambah. Pada percobaan ini, LSTM dua layer memiliki 50.497 parameter dibanding 17.217 pada satu layer, dan waktu trainingnya naik dari 0,111731 menjadi 0,167531 detik. MSE testing justru naik dari 0,000205 menjadi 0,001253 pada 50 epoch karena model yang lebih dalam konvergen lebih lambat. Untuk data sinus dan cosinus yang sederhana, layer tambahan tidak memberi manfaat.

BAB V

PENUTUP

5.1 Kesimpulan

LSTM berhasil digunakan untuk memprediksi deret sinus dan cosinus dengan sequence length 20. Pada data sinus, LSTM memperoleh MSE testing 0,000282 dan RNN memperoleh 0,000364 dengan konfigurasi yang sama. LSTM sedikit lebih baik dan kurva loss-nya lebih mulus, tetapi selisihnya kecil dan berasal dari satu percobaan sehingga keunggulan LSTM belum dapat dipastikan pada data sederhana ini. Hidden size yang lebih besar menurunkan MSE testing LSTM dari 0,000647 pada hidden size 32 menjadi 0,000282 pada hidden size 64 dan 0,000211 pada hidden size 128, dengan waktu training yang naik dari 0,137526 menjadi 0,418476 detik.

Pada tugas cosinus, LSTM_2 dengan hidden size 64 dan satu layer memberi hasil terbaik dengan MSE testing 0,000205. Penambahan menjadi dua layer pada LSTM_3 menaikkan MSE testing menjadi 0,001253 pada 50 epoch dan menambah waktu training, sehingga layer yang lebih banyak tidak menjamin hasil yang lebih baik pada data dan pengaturan pelatihan ini.

DAFTAR PUSTAKA

- [1] Goodfellow, I., Bengio, Y., dan Courville, A. 2016. Deep Learning. Cambridge: MIT Press.
- [2] Elman, J. L. 1990. Finding Structure in Time. *Cognitive Science*, 14(2), 179-211. https://doi.org/10.1207/s15516709cog1402_1.
- [3] Hochreiter, S. 1998. The Vanishing Gradient Problem During Learning Recurrent Neural Nets and Problem Solutions. *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, 6(2), 107-116. <https://doi.org/10.1142/S0218488598000094>.
- [4] Hochreiter, S., dan Schmidhuber, J. 1997. Long Short-Term Memory. *Neural Computation*, 9(8), 1735-1780. <https://doi.org/10.1162/neco.1997.9.8.1735>.
- [5] Gers, F. A., Schmidhuber, J., dan Cummins, F. 2000. Learning to Forget: Continual Prediction with LSTM. *Neural Computation*, 12(10), 2451-2471. <https://doi.org/10.1162/089976600300015015>.
- [6] Kingma, D. P., dan Ba, J. 2015. Adam: A Method for Stochastic Optimization. *International Conference on Learning Representations*. arXiv:1412.6980.
- [7] Paszke, A., et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. *Advances in Neural Information Processing Systems*, 32.
- [8] Modul Praktikum Deep Learning Minggu 4. 2026. Long Short-Term Memory (LSTM) untuk Prediksi Deret Waktu. Program Studi Informatika, Universitas Andalas.